Agent Design Document

Project: Student Information Management System (SIMS) **Student:** [Student name]

> Honesty note: this project was built with **one** Claude Code agent session (Claude Sonnet 5.5). No parallel sub-agents were spawned. The "agents" below are **roles the single agent performed in sequence**, each with its own inputs, outputs and verification step. This keeps the design truthful while still showing specialised responsibilities. If you ran separate tools/agents yourself, document them here instead.

1. Architecture

```
Student (human, decision maker)
   |  prompt
   v
Planner role ──> work breakdown + requirement checklist
   |
   ├── Database role     -> migrations, relationships, factories, seeder
   ├── Laravel role      -> routes, controllers, Form Requests, middleware, API resources
   ├── UI role           -> Blade layout, components (data-table, action-button),
pagination view
   ├── Testing/Review role -> PHPUnit feature tests, audit against rubric, security review
   └── Documentation role  -> README, OpenAPI, SQL export, screenshots, these PDFs
   |
   v
Student reviews output, corrects direction, approves
```

2. Roles and responsibilities

Role	Responsible for	Inputs	Outputs	Verification
Planner	Requirements analysis, work breakdown	Handbook PDF, rubric PDF, bonus txt	Feature checklist, order of work	Re-checked against rubric during the audit
Database	Schema, FK, unique constraints, relationships	Planner checklist	4 migrations, 3 models, factories, seeder	`migrate:fresh --seed`; cascade-delete test

Role	Responsible for	Inputs	Outputs	Verification
Laravel	MVC code, validation, authorisation, API	Schema, checklist	Controllers, 6 Form Requests, `admin` middleware, `StudentResource`, Sanctum API	Feature tests (web + API)
UI	Layout, tables, pagination, mobile	User UI prompts	Sidebar layout, `x-data-table`, `x-action-button`, custom pager	Rendered pages checked via screenshots / HTML output
Testing & Review	Bugs, validation review, security review	All code	32 tests; audit report	`php artisan test`
Documentation	README, API docs, reports	Final code	README, `openapi.yaml`, `sims_export.sql`, PDFs	Restore test of SQL export; YAML parse check

3. Context management

- **Feature-based context.** Work was split by feature (auth -> students -> subjects -> enrollments -> API -> UI), so each step only needed the files of that feature.
- **Source of truth up front.** The three requirement documents were read once; requirements were distilled into a checklist that was reused instead of re-reading the PDFs.
- **Context reuse through code.** Repeated UI patterns became Blade components (`x-data-table`, `x-action-button`) and shared Form Requests (Update extends Store). A change in one place updates all three modules, so fewer files had to be re-sent or re-explained.
- **State on disk, not in the chat.** Tests, README and migrations are the persistent memory; later prompts relied on them instead of re-describing the system.

4. Token management

Strategy	How it was applied
Task decomposition	One concern per prompt (sidebar, header, pagination, icons, mobile)
Incremental generation	Large batches of files failed in the shell (see reflection); the agent split them into small writes
Targeted reads	PDF pages and file ranges were read selectively (`limit`, specific edits) rather than whole-file dumps
Small edits over rewrites	CSS tweaks (borders, max-width) were single-string edits, not full-file rewrites
Specialised roles	Each role only needs its own inputs (e.g. UI role never needs the migrations)

5. Handbook reflection questions

1. **How did you reduce unnecessary AI context?** Feature-by-feature work, components for repeated UI, targeted edits.
2. **How did you avoid prompt repetition?** Requirements were captured once as a checklist; shared components and base classes removed repeated instructions.
3. **How did you verify generated outputs?** `php artisan test` after each change (32 tests), route listing, rendering real pages, restoring the SQL export into an empty database, parsing the OpenAPI file.
4. **How did you balance token cost and quality?** Cheap checks (tests, grep, route list) ran after every change; expensive full rewrites were avoided.